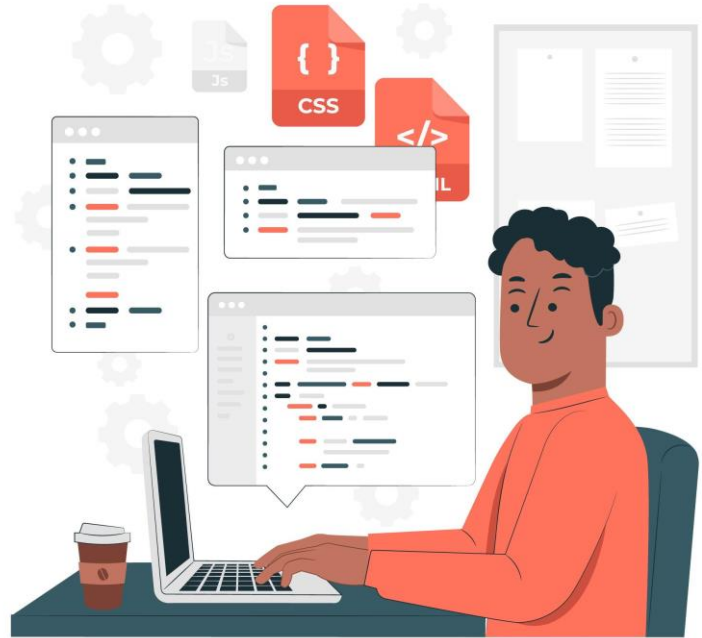

REST API dengan Raw MySQL





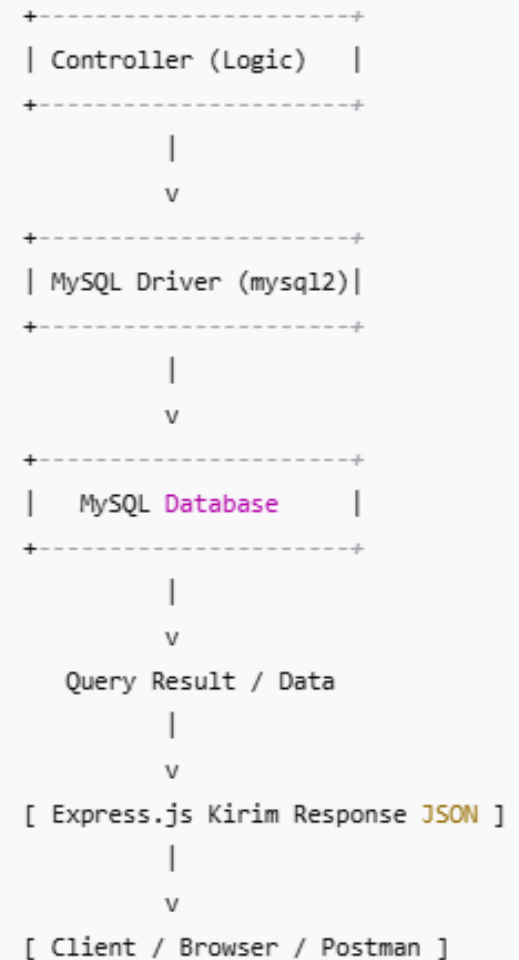
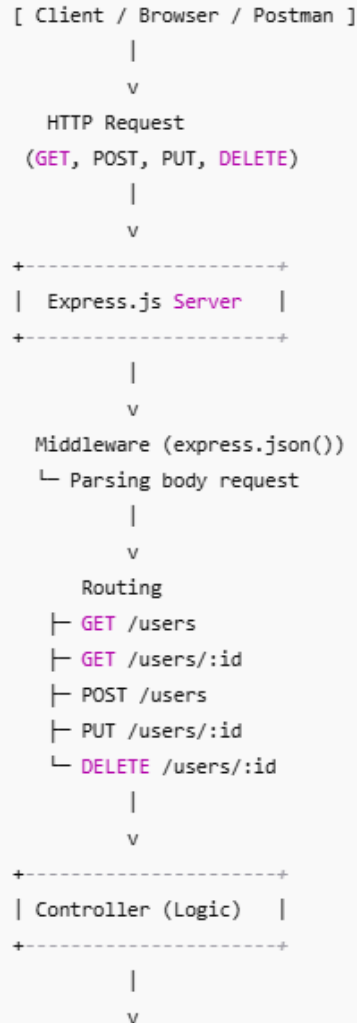
KEMNAKER

Outline

- Koneksi database dengan MySQL
- CRUD REST API
 - Create
 - Read
 - Update
 - Delete

Alur

- **JSON** → Format komunikasi antara client & server. JSON berperan sebagai **format data di jalur komunikasi** antara *client* dan *server*, **bukan** format penyimpanan di database.
- **MySQL** → Tempat penyimpanan permanen di server.
- REST API bertugas **mengubah data MySQL jadi JSON saat dikirim** dan **mengubah JSON jadi query SQL saat menerima**.



Koneksi database dengan MySQL

Install package MySQL?

agar bisa membuat koneksi database dengan MySQL hal pertama yang dilakukan adalah install package MySQL terlebih dahulu <https://github.com/mysqljs/mysql>

package tersebut bisa dapat di-install menggunakan perintah

npm install mysql

DEMO

CRUD REST API

Struktur Folder Project

```
project-folder/  
├ .env  
├ package.json  
├ app.js  
└ src/  
  ├── config/  
  │   └ db.js  
  ├── controllers/  
  │   └ movieController.js  
  ├── routes/  
  │   └ movieRoutes.js  
  └ middleware/  
      └ validation.js
```

1) SQL: buat database & tabel movies

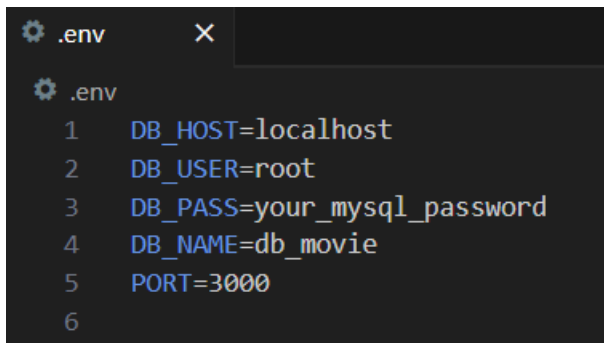
```
CREATE DATABASE IF NOT EXISTS db_movie;  
USE db_movie;
```

```
CREATE TABLE IF NOT EXISTS movies (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(255) NOT NULL,  
    year INT NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

Perhatikan penulisan → case sensitive

2) File .env (contoh)

Buat file .env di root project:



```
.env
1 DB_HOST=localhost
2 DB_USER=root
3 DB_PASS=your_mysql_password
4 DB_NAME=db_movie
5 PORT=3000
6
```

Ganti your_mysql_password sesuai password MySQL Anda.

Fungsi menempatkan DB_PASS di file .env

- **Memisahkan konfigurasi dari kode**
Password, username, host, dll tidak ditulis langsung di source code (misalnya db.js), tapi dipanggil dari .env.
→ Kode jadi lebih **bersih** dan lebih **mudah dipindahkan** ke environment lain (local, staging, production).
- **Memudahkan perubahan**
Misal di lokal pakai user root tanpa password, tapi di server pakai user khusus dengan password kuat.
→ Cukup ubah file .env, tidak perlu ubah kode.
- **Mencegah kebocoran di GitHub/repository**
.env biasanya dimasukkan ke dalam .gitignore, sehingga **tidak ikut ter-upload ke repo publik**.
→ Password tidak akan terlihat oleh siapa pun yang mengakses repository Anda.

Fungsi .env

1. **Konfigurasi terpusat** → semua setting ada di .env.
 2. **Keamanan** → file .env tidak perlu di-commit ke Git.
 3. **Fleksibel** → ganti environment tanpa edit kode.
 4. **Best practice** → hampir semua project Node.js/Express besar memakainya.
-
- Menyimpan **konfigurasi sensitif** atau **nilai yang sering berubah** (misalnya password, API key, nama database, port server) di luar kode.
 - Memudahkan pengaturan environment antara **development**, **testing**, dan **production** tanpa harus ubah kode.
 - Mencegah **kredensial sensitif** ikut masuk ke repository Git (tinggal masukkan .env ke .gitignore).

3) package.json (instal dependensi)

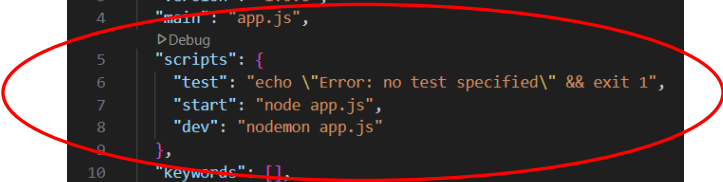
Inisialisasi dan instal:

```
npm init -y  
npm install express mysql2 dotenv
```

```
# (opsional untuk development)  
npm install --save-dev nodemon
```

Tambahkan script di package.json (opsional):

```
"scripts": {  
  "start": "node app.js",  
  "dev": "nodemon app.js"  
}
```

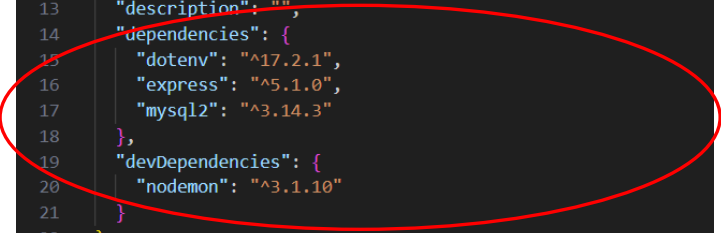


```
LatExpressRawMySQL > {} package.json > ...  
1 {  
2   "name": "latexpressrawmysql",  
3   "version": "1.0.0",  
4   "main": "app.js",  
5   "scripts": {  
6     "test": "echo \\\"Error: no test specified\\\" && exit 1",  
7     "start": "node app.js",  
8     "dev": "nodemon app.js"  
9   },  
10  "keywords": [],  
11  "author": "",  
12  "license": "ISC",  
13  "description": "",  
14  "dependencies": {  
15    "dotenv": "^17.2.1",  
16    "express": "^5.1.0",  
17    "mysql2": "^3.14.3"  
18  },  
19  "devDependencies": {  
20    "nodemon": "^3.1.10"  
21  }  
22 }  
23
```

3) package.json (instal dependensi)

Setelah inisialisasi & instal, maka package.json akan otomatis terupdate sendiri :

```
LatExpressRawMySQL > {} package.json > ...
1  {
2    "name": "latexpressrawmysql",
3    "version": "1.0.0",
4    "main": "app.js",
5    >Debug
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1",
8      "start": "node app.js",
9      "dev": "nodemon app.js"
10   },
11   "keywords": [],
12   "author": "",
13   "license": "ISC",
14   "description": "",
15   "dependencies": {
16     "dotenv": "^17.2.1",
17     "express": "^5.1.0",
18     "mysql2": "^3.14.3"
19   },
20   "devDependencies": {
21     "nodemon": "^3.1.10"
22   }
23 }
```



3) package.json (instal dependensi)...Penjelasan

npm init -y

Perintah `npm init -y` digunakan untuk menginisialisasi proyek Node.js dengan cepat dan mudah, dengan mengisi semua opsi secara otomatis dengan nilai default. Ini adalah cara pintas untuk `npm init` yang akan meminta pengguna untuk mengisi setiap opsi secara manual.

Secara rinci, `npm init -y` melakukan hal-hal berikut:

1. Membuat file package.json:

File ini berisi metadata tentang proyek Node.js Anda, seperti nama, versi, deskripsi, dependensi, dan skrip.

2. Mengisi nilai default:

Secara otomatis mengisi semua opsi yang biasanya diminta oleh `npm init` dengan nilai default, seperti "1.0.0" untuk versi, "test" untuk perintah pengujian, dan sebagainya.

Dengan kata lain, `npm init -y` adalah cara cepat untuk membuat file `package.json` yang siap digunakan dengan nilai-nilai umum yang sering digunakan dalam proyek Node.js.

3) package.json (instal dependensi)...Penjelasan

npm init -y

```
{ } package.json > ...
1  {
2    "name": "latexpressrawmysql",
3    "version": "1.0.0",
4    "main": "app.js",
   > Debug
5    "scripts": {
6      "test": "echo \"Error: no test specified\" && exit 1"
7    },
8    "keywords": [],
9    "author": "",
10   "license": "ISC",
11   "description": ""
12 }
```

3) package.json (instal dependensi)...Penjelasan

```
npm install express mysql2 dotenv
```

Install express

Install mysql yang versi terbaru → mysql2

Install dotenv

Dotenv adalah **library Node.js** yang tugasnya membaca file .env dan memasukkan isinya ke dalam process.env di aplikasi kamu.

Kenapa harus diinstal sendiri?

Node.js default-nya tidak bisa membaca .env file.

Node cuma menyediakan process.env untuk baca variable environment, tapi untuk memuatnya dari file .env, butuh bantuan library.

dotenv adalah library populer dan ringan yang dibuat khusus untuk itu.

3) package.json (instal dependensi)...Penjelasan

```
# (optional untuk development)
npm install --save-dev nodemon
```

nodemon adalah sebuah tool yang sangat membantu saat kamu sedang mengembangkan aplikasi **Node.js**. Fungsinya adalah untuk **memantau perubahan file dalam proyek** dan secara otomatis **me-restart server** ketika ada perubahan.

Penjelasan singkat:

- Nama: nodemon = **Node Monitor**
- Fungsi: Auto-restart aplikasi Node.js saat ada perubahan file.
- Cocok digunakan saat development agar tidak perlu jalankan ulang manual setiap kali mengedit kode.

nodemon

Contoh penggunaan:

Misal kamu punya file `app.js`, jalankan dengan:

```
bash
```

```
npx nodemon app.js
```

Atau kalau kamu sudah install secara global:

```
bash
```

```
nodemon app.js
```

nodemon

Instalasi:

Secara global (supaya bisa dipakai di mana saja):

```
bash  
  
npm install -g nodemon
```

Secara lokal (hanya dalam proyek):

```
bash  
  
npm install --save-dev nodemon
```

Kemudian bisa tambahkan di `package.json` :

```
json  
  
"scripts": {  
  "start": "nodemon app.js"  
}
```

nodemon

Kelebihan nodemon:

- Otomatis reload saat ada perubahan file .js, .json, atau file lain yang kamu tentukan.
- Bisa dikustomisasi (misalnya hanya monitor folder tertentu).

nodemon $\approx$ "hot reload"-nya backend.

Bedanya, nodemon restart server secara utuh.

Sementara hot reload di frontend biasanya cuma mengganti bagian kode yang berubah tanpa hilangkan state aplikasi.

Hot reload ?

Hot Reload adalah fitur dalam pengembangan aplikasi (biasanya frontend) yang memungkinkan perubahan kode langsung terlihat di layar tanpa me-refresh halaman atau kehilangan data yang sedang ditampilkan (state).

4) Kode: app.js (utama)

Buat file app.js di root:

semua route yang didefinisikan di movieRoutes.js akan otomatis dipasang di bawah prefix /api/movies.

Penjelasan per baris (app.js)

1. `require('dotenv').config();` — load variabel lingkungan dari `.env`.
2. `const express = require('express');` — import Express.
3. `const app = express();` — buat instance Express.
4. `const port = process.env.PORT || 3000;` — ambil port dari env atau default 3000.
5. `require('./src/config/db');` — memuat konfigurasi DB (menjalankan test koneksi).
6. `const movieRoutes = require('./src/routes/movieRoutes');` — load routes movies.
7. `app.use(express.json());` — middleware global untuk JSON body parsing.
8. `app.use('/api/movies', movieRoutes);` — mount route dengan prefix `/api/movies`.
9. `app.use((req, res) => { ... });` — fallback 404 response.
10. `app.listen(port, () => { ... });` — jalankan server dan log port.

```
// app.js
require('dotenv').config(); // Load .env
const express = require('express');
const app = express();
const port = process.env.PORT || 3000;

// inisialisasi DB (kita hanya require sehingga file src/config/db.js dijalankan)
require('./src/config/db');

const movieRoutes = require('./src/routes/movieRoutes');

// middleware untuk parsing JSON body
app.use(express.json());

// prefix routes sesuai permintaan
app.use('/api/movies', movieRoutes);

// simple 404 handler untuk route yang tidak ada
app.use((req, res) => {
  res.status(404).json({ message: 'Endpoint not found' });
});

// start server
app.listen(port, () => {
  console.log(`Server berjalan pada port ${port}`);
});
```

5) Kode: src/config/db.js (konfigurasi MySQL, callback style)

Buat file src/config/db.js:

```
// src/config/db.js
const mysql = require('mysql');
require('dotenv').config(); // untuk memastikan env ter-load jika dipanggil sendiri

const pool = mysql.createPool({
  connectionLimit: 10,
  host: process.env.DB_HOST || 'localhost',
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASS || '',
  database: process.env.DB_NAME || 'db_movie'
});

// cek koneksi saat aplikasi mulai
pool.getConnection((err, connection) => {
  if (err) {
    console.error('Gagal konek ke database MySQL:', err.message);
    // jangan throw agar server masih bisa jalan (opsional: Anda bisa proses.exit jika ingin)
    return;
  }
  console.log('Terhubung ke MySQL (pool) - siap menerima query');
  connection.release(); // kembalikan koneksi ke pool
});

module.exports = pool;
```

Penjelasan per baris (db.js)

- `const mysql = require('mysql');` — import package `mysql` (callback style).
- `require('dotenv').config();` — pastikan `.env` ter-load (keamanan ganda).
- `const pool = mysql.createPool({...});` — buat connection pool menggunakan env vars.
 - `connectionLimit: 10` — maksimal koneksi simultan.
 - `host, user, password, database` diambil dari `.env` atau default.
- `pool.getConnection((err, connection) => { ... });` — ambil 1 koneksi dari pool untuk cek koneksi saat app start.
- Jika error -> log error.
- Jika sukses -> log dan `connection.release()` kembalikan koneksi ke pool.
- `module.exports = pool;` — ekspor pool agar controller bisa gunakan.

Membuat Koneksi Database dengan MySQL

```
const mysql = require("mysql")

const connectionPool = mysql.createPool({
  host      : 'localhost',
  user      : 'root',
  password  : 'password',
  database  : 'db_movie'
});

connectionPool.getConnection(function(err) {
  if (err) {
    throw err
  }
});

module.exports = {
  connectionPool
}
```

callback

```
function callback(err, result) {  
  if (err) {  
    // Tangani error  
  } else {  
    // Gunakan result  
  }  
}
```

```
// cek koneksi saat aplikasi mulai  
pool.getConnection((err, connection) => {  
  if (err) {  
    console.error('Gagal konek ke database MySQL:', err.message);  
    // jangan throw agar server masih bisa jalan (opsional: Anda bisa proses.exit jika ingin)  
    return;  
  }  
  console.log('Terhubung ke MySQL (pool) – siap menerima query');  
  connection.release(); // kembalikan koneksi ke pool  
});
```

Versi modern perpaduan callback, promise, async/await

```
const mysql = require('mysql2');

const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'testdb'
});

connection.query('SELECT * FROM users', (err, results) => {
  if (err) throw err;
  console.log(results);
});

connection.end();
```

```
const mysql = require('mysql2/promise');

async function getUsers() {
  const connection = await mysql.createConnection({
    host: 'localhost',
    user: 'root',
    password: '',
    database: 'testdb'
  });

  const [rows] = await connection.execute('SELECT * FROM users');
  console.log(rows);
  await connection.end();
}

getUsers();
```

Membuat Koneksi Database dengan MySQL

jika DB sudah diset di .env maka nanti edit db.js :

db.js

.env

```
.env
1 DB_HOST=localhost
2 DB_USER=root
3 DB_PASS=your_mysql_password
4 DB_NAME=db_movie
5 PORT=3000
6
```

```
LatExpressRawMySQL > src > config > JS db.js > ...
1 // src/config/db.js
2 const mysql = require('mysql');
3 require('dotenv').config(); // untuk memastikan env ter-load jika dipanggil sendiri
4
5 const pool = mysql.createPool({
6   connectionLimit: 10,
7   host: process.env.DB_HOST || 'localhost',
8   user: process.env.DB_USER || 'root',
9   password: process.env.DB_PASS || '',
10  database: process.env.DB_NAME || 'db_movie'
11 });
12
13 // cek koneksi saat aplikasi mulai
14 pool.getConnection((err, connection) => {
15   if (err) {
16     console.error('Gagal konek ke database MySQL:', err.message);
17     // jangan throw agar server masih bisa jalan (opsional: Anda bisa proses.exit jika ingin)
18     return;
19   }
20   console.log('Terhubung ke MySQL (pool) - siap menerima query');
21   connection.release(); // kembalikan koneksi ke pool
22 });
23
24 module.exports = pool;
```

6) Middleware validasi: src/middleware/validation.js

Buat file
src/middleware/validation.js:

Penjelasan per baris (validation.js)

1. Ekspor fungsi middleware `validateMovie`.
2. Ambil `title` dan `year` dari `req.body`.
3. Cek `title` kosong -> balas 400.
4. Cek `year` kosong/null/undefined -> balas 400.
5. Opsional: parse year jadi angka, jika NaN balas 400.
6. Jika valid -> panggil `next()` agar lanjut ke controller.

Middleware ini dipasang untuk route POST & PUT sehingga `title` & `year` tidak boleh kosong.

LatExpressRawMySQL > src > middleware > JS validation.js > ...

```
1 // src/middleware/validation.js
2 module.exports = function validateMovie(req, res, next) {
3   const { title, year } = req.body;
4
5   if (!title || title.toString().trim() === '') {
6     return res.status(400).json({ error: 'Title harus diisi' });
7   }
8
9   if (year === undefined || year === null || year.toString().trim() === '') {
10    return res.status(400).json({ error: 'Year harus diisi' });
11  }
12
13  // opsional: cek apakah year numeric
14  const yearNum = parseInt(year, 10);
15  if (isNaN(yearNum)) {
16    return res.status(400).json({ error: 'Year harus berupa angka' });
17  }
18
19  // semua lolos
20  next();
21 };
```

7) Controller: src/controllers/movieController.js

Buat file src/controllers/movieController.js:

```
LatExpressRawMySQL > src > controllers > JS movieController.js > ...  
1  // src/controllers/movieController.js  
2  const pool = require('../config/db');  
3
```

Catatan : Semua operasi menggunakan callback style (pool.getConnection + conn.query)

Callback di controller

Callback di controller = fungsi (err, result) => {...} yang dipakai setelah operasi database selesai.

Di tiap controller (create, read, update, delete), minimal ada **2 callback**:

- Saat ambil koneksi (getConnection)
- Saat jalankan query (conn.query)

db.getConnection((err, conn) => {...})

- Callback ini dipanggil setelah pool mencoba kasih koneksi.
- Kalau error (misalnya DB mati), err akan terisi.
- Kalau berhasil, kita dapat objek conn.

conn.query(sql, values, (err, results) => {...})

- Callback ini dipanggil setelah query selesai dijalankan.
- Kalau query salah, err akan berisi pesan error dari MySQL.
- Kalau query sukses, results berisi data hasil query (array of row object).

CREATE

```
exports.createMovie = (req, res) => {
  const { title, year } = req.body;

  pool.getConnection((err, conn) => {
    if (err) {
      return res.status(500).json({ error: 'DB connection error', detail: err.message });
    }

    const sqlInsert = 'INSERT INTO movies (title, year) VALUES (?, ?)';
    conn.query(sqlInsert, [title, year], (errInsert, result) => {
      if (errInsert) {
        conn.release();
        return res.status(500).json({ error: 'DB insert error', detail: errInsert.message });
      }
    });
  });
};
```

CREATE

Tambahkan kode
untuk menampilkan
hasil insert

LatExpressRawMySQL > src > controllers > JS movieController.js > ...

```
4  /**
5   * CREATE movie
6   * (POST /api/movies)
7   */
8  exports.createMovie = (req, res) => {
9    const { title, year } = req.body;
10
11    pool.getConnection((err, conn) => {
12      if (err) {
13        return res.status(500).json({ error: 'DB connection error', detail: err.message });
14      }
15
16      const sqlInsert = 'INSERT INTO movies (title, year) VALUES (?, ?)';
17      conn.query(sqlInsert, [title, year], (errInsert, result) => {
18        if (errInsert) {
19          conn.release();
20          return res.status(500).json({ error: 'DB insert error', detail: errInsert.message });
21        }
22
23        const newId = result.insertId;
24        // ambil record yang baru dibuat untuk dikembalikan lengkap dengan timestamp
25        const sqlSelect = 'SELECT * FROM movies WHERE id = ?';
26        conn.query(sqlSelect, [newId], (errSelect, rows) => {
27          conn.release();
28          if (errSelect) {
29            return res.status(500).json({ error: 'DB select error', detail: errSelect.message });
30          }
31          res.status(201).json(rows[0]);
32        });
33      });
34    });
35  };
```

Callback digunakan **berlapis 3 kali**:

CALLBACK pada CREATE

JavaScript

```
pool.getConnection((err, conn) => { // callback #1
  conn.query(sqlInsert, [title, year], (errInsert, result) => { // callback #2
    conn.query(sqlSelect, [newId], (errSelect, rows) => { // callback #3
      res.status(201).json(rows[0]);
    });
  });
});
```

Penjelasan:

- Callback #1 → menangani hasil peminjaman koneksi.
- Callback #2 → menangani hasil query INSERT.
- Callback #3 → menangani hasil query SELECT setelah insert berhasil.
- Semua nested, karena query berikutnya baru bisa dijalankan setelah query sebelumnya sukses.

Jenis:

- **Callback chaining / nested callback** (kadang disebut *callback hell* kalau terlalu dalam).

READ

LatExpressRawMySQL > src > controllers > JS movieController.js > ...

```
37  /**
38  * READ all movies
39  * (GET /api/movies)
40  */
41  exports.getAllMovies = (req, res) => {
42    pool.getConnection((err, conn) => {
43      if (err) return res.status(500).json({ error: 'DB connection error', detail: err.message });
44
45      const sql = 'SELECT * FROM movies ORDER BY id DESC';
46      conn.query(sql, (errQuery, rows) => {
47        conn.release();
48        if (errQuery) return res.status(500).json({ error: 'DB query error', detail: errQuery.message });
49        res.json(rows);
50      });
51    });
52  };
53
```

UPDATE

LatExpressRawMySQL > src > controllers > JS movieController.js > ...

```
76  * UPDATE movie
77  * (PUT /api/movies/:id)
78  */
79  exports.updateMovie = (req, res) => {
80    const id = parseInt(req.params.id, 10);
81    const { title, year } = req.body;
82    if (isNaN(id)) return res.status(400).json({ error: 'ID tidak valid' });
83
84    pool.getConnection((err, conn) => {
85      if (err) return res.status(500).json({ error: 'DB connection error', detail: err.message });
86
87      const sqlUpdate = 'UPDATE movies SET title = ?, year = ?, updated_at = NOW() WHERE id = ?';
88      conn.query(sqlUpdate, [title, year, id], (errUpdate, result) => {
89        if (errUpdate) {
90          conn.release();
91          return res.status(500).json({ error: 'DB update error', detail: errUpdate.message });
92        }
93
94        if (result.affectedRows === 0) {
95          conn.release();
96          return res.status(404).json({ error: 'Movie tidak ditemukan' });
97        }
98
99        // ambil record yang sudah diupdate
100      const sqlSelect = 'SELECT * FROM movies WHERE id = ?';
101      conn.query(sqlSelect, [id], (errSelect, rows) => {
102        conn.release();
103        if (errSelect) return res.status(500).json({ error: 'DB select error', detail: errSelect.message });
104        res.json(rows[0]);
105      });
106    });
107  });
108  };
```

DELETE

LatExpressRawMySQL > src > controllers > JS movieController.js > ...

```
110  /**
111   * DELETE movie
112   * (DELETE /api/movies/:id)
113   */
114  exports.deleteMovie = (req, res) => {
115    const id = parseInt(req.params.id, 10);
116    if (isNaN(id)) return res.status(400).json({ error: 'ID tidak valid' });
117
118    pool.getConnection((err, conn) => {
119      if (err) return res.status(500).json({ error: 'DB connection error', detail: err.message });
120
121      const sql = 'DELETE FROM movies WHERE id = ?';
122      conn.query(sql, [id], (errDel, result) => {
123        conn.release();
124        if (errDel) return res.status(500).json({ error: 'DB delete error', detail: errDel.message });
125        if (result.affectedRows === 0) return res.status(404).json({ error: 'Movie tidak ditemukan' });
126        res.json({ message: 'Movie berhasil dihapus' });
127      });
128    });
129  };
130
```

8) Route file:

Buat file
src/routes/movieRoutes.js:

- GET / → getAllMovies
- GET /:id → getMovieById
- POST / → validateMovie → createMovie
- PUT /:id → validateMovie → updateMovie
- DELETE /:id → deleteMovie

LatExpressRawMySQL > src > routes > JS movieRoutes.js > ...

```
1 // src/routes/movieRoutes.js
2 const express = require('express');
3 const router = express.Router();
4
5 const movieController = require('../controllers/movieController');
6 const validateMovie = require('../middleware/validation');
7
8 // READ all
9 router.get('/', movieController.getAllMovies);
10
11 // READ one
12 router.get('/:id', movieController.getMovieById);
13
14 // CREATE (gunakan middleware validasi)
15 router.post('/', validateMovie, movieController.createMovie);
16
17 // UPDATE (gunakan middleware validasi)
18 router.put('/:id', validateMovie, movieController.updateMovie);
19
20 // DELETE
21 router.delete('/:id', movieController.deleteMovie);
22
23 module.exports = router;
```

9) Endpoint

CREATE / READ / UPDATE / DELETE

- **CREATE:** `exports.createMovie` di `movieController.js` — endpoint `POST /api/movies`.
- **READ (semua):** `exports.getAllMovies` — endpoint `GET /api/movies`.
- **READ (satu):** `exports.getMovieById` — endpoint `GET /api/movies/:id`.
- **UPDATE:** `exports.updateMovie` — endpoint `PUT /api/movies/:id`.
- **DELETE:** `exports.deleteMovie` — endpoint `DELETE /api/movies/:id`.

10. Running

```
npm install  
npm run dev    # jika pakai nodemon  
# atau  
npm start
```

Jika berhasil →

```
$ npm run dev  
  
> latexpressrawmysql@1.0.0 dev  
> nodemon app.js  
  
[nodemon] 3.1.10  
[nodemon] to restart at any time, enter `rs`  
[nodemon] watching path(s): *.*  
[nodemon] watching extensions: js,mjs,cjs,json  
[nodemon] starting `node app.js`  
[dotenv@17.2.1] injecting env (5) from .env -- tip: 📦 auto-backup env with Radar: https://dotenvx.com/radar  
[dotenv@17.2.1] injecting env (0) from .env -- tip: 📦 version env with Radar: https://dotenvx.com/radar  
Server berjalan pada port 3000  
Terhubung ke MySQL (pool) – siap menerima query  
□
```

11. Pengujian / Testing

Misalkan menggunakan thunder client di VSCode

Asumsikan server berjalan di `http://localhost:3000`.

1) Create movie

- Method: `POST`
- URL: `http://localhost:3000/api/movies`
- Headers: `Content-Type: application/json`
- Body (JSON):

```
json

{
  "title": "Inception",
  "year": 2010
}
```

- Expected Response: `201 Created`

```
json

{
  "id": 1,
  "title": "Inception",
  "year": 2010,
  "created_at": "2025-08-21T...Z",
  "updated_at": "2025-08-21T...Z"
}
```

11. Pengujian / Testing

Misalkan menggunakan thunder client di VSCode

Asumsikan server berjalan di `http://localhost:3000`.

2) Get all movies

- Method: `GET`
- URL: `http://localhost:3000/api/movies`
- Expected Response: `200 OK` (array of movies)

3) Get single movie

- Method: `GET`
- URL: `http://localhost:3000/api/movies/1`
- Expected Response: `200 OK` movie object (atau `404` jika id tidak ada)

11. Pengujian / Testing

Misalkan menggunakan thunder client di VSCode

Asumsikan server berjalan di `http://localhost:3000`.

4) Update movie

- Method: `PUT`
- URL: `http://localhost:3000/api/movies/1`
- Headers: `Content-Type: application/json`
- Body:

json

```
{
  "title": "Inception (Updated)",
  "year": 2010
}
```

- Expected Response: `200 OK` updated movie object (updated_at berubah)

11. Pengujian / Testing

Misalkan menggunakan thunder client di VSCode

Asumsikan server berjalan di `http://localhost:3000`.

5) Delete movie

- Method: `DELETE`
- URL: `http://localhost:3000/api/movies/1`
- Expected Response: `200 OK`

json

```
{ "message": "Movie berhasil dihapus" }
```

12. Sebelum upload ke GitHub

Buat file .gitignore yang isinya :

- node_modules
- .env

Agar 2 file tsb tdk terupload di github.

12. Sebelum upload ke GitHub

a) `node_modules/`

- Folder ini berisi **ribuan file library** hasil install dari `npm install`.
- Beratnya bisa ratusan MB.
- Semua dependency sudah tertulis di `package.json` dan `package-lock.json`.
- Jadi kalau project dipindah ke server/komputer lain, cukup jalankan:

```
nginx  
  
npm install
```

→ otomatis semua library akan ter-install ulang.

- Kalau ikut di-push ke GitHub → repository jadi **sangat besar** & tidak efisien.

b) `.env`

- File ini berisi **data sensitif** seperti:
 - username database
 - password database
 - API key
 - secret token
- Kalau `.env` ikut ter-upload ke GitHub:
 - Orang lain bisa melihat **password/database/API key Anda**
 - Bisa dipakai untuk **hack database** atau **menyalahgunakan API Anda**
 - Ini **sangat berbahaya** kalau repo Anda public

TAMBAHAN

**versi Express REST API
dengan mysql2/promise (async/await)
untuk CRUD Movies**

Struktur folder tetap sama

.env sama

src/routes/movieRoutes.js sama

App.js sama

Uninstall mysql lalu instal lagi npm install mysql2



Edit src/config/db.js

Menggunakan **versi driver MySQL yang berbasis Promise**

```
const mysql = require('mysql2/promise');
```

- Kalau pakai mysql2 biasa → callback.
- Kalau pakai mysql2/promise → setiap function seperti query(), execute(), getConnection() **langsung mengembalikan Promise.**

Edit src/config/db.js

```
pool = await mysql.createPool({  
  host: process.env.DB_HOST,  
  user: process.env.DB_USER,  
  password: process.env.DB_PASS,  
  database: process.env.DB_NAME  
});
```

➡ `mysql.createPool()` versi `mysql2/promise` mengembalikan Promise juga.

Makanya kita bungkus dengan `await`.

Edit src/config/db.js

LatExpressRawMySQLPromiseAsync > src > config > JS db.js > ...

```
1  // src/config/db.js
2  const mysql = require('mysql2/promise'); // gunakan versi promise
3  require('dotenv').config();
4
5  let pool;
6
7  (async () => {
8    try {
9      pool = await mysql.createPool({
10        connectionLimit: 10,
11        host: process.env.DB_HOST || 'localhost',
12        user: process.env.DB_USER || 'root',
13        password: process.env.DB_PASS || '',
14        database: process.env.DB_NAME || 'db_movie'
15      });
16
17      console.log('Terhubung ke MySQL (pool) – siap menerima query (Promise API)');
18    } catch (err) {
19      console.error('Gagal konek ke database MySQL:', err.message);
20    }
21  })();
22
23  module.exports = () => pool;
```

Edit src/controllers/movieController.js

```
const [rows] = await pool.query('SELECT * FROM movies');
```

➡ `pool.query(...)` itu sebenarnya **Promise**.

Kalau ditulis tanpa `await`, bentuknya seperti ini:

```
pool.query('SELECT * FROM movies')
  .then(([rows]) => {
    console.log(rows);
  })
  .catch(err => {
    console.error(err);
  });
```

Tapi karena kita pakai **async/await**, maka:

```
const [rows] = await pool.query('SELECT * FROM movies');
```

lebih ringkas daripada `.then().catch()`.

Edit
src/controllers/
movieController.
js

```
LatExpressRawMySQLPromiseAsync > src > controllers > JS movieController.js > ...
1 //Karena kita sekarang pakai Promise API, controller bisa ditulis dengan async/await:
2
3 // src/controllers/movieController.js
4 const getPool = require('../config/db');
5
6 // CREATE
7 exports.createMovie = async (req, res) => {
8   try {
9     const pool = getPool();
10    const { title, year } = req.body;
11
12    if (!title || !year) {
13      return res.status(400).json({ error: 'Title dan Year wajib diisi' });
14    }
15
16    const [result] = await pool.query(
17      'INSERT INTO movies (title, year, created_at, updated_at) VALUES (?, ?, NOW(), NOW())',
18      [title, year]
19    );
20
21    const newId = result.insertId;
22    const [rows] = await pool.query('SELECT * FROM movies WHERE id = ?', [newId]);
23
24    res.status(201).json(rows[0]);
25  } catch (err) {
26    res.status(500).json({ error: err.message });
27  }
28 };
29
```

Edit
src/controllers/
movieController.
js

LatExpressRawMySQLPromiseAsync > src > controllers > JS movieController.js > ...

```
30 // READ ALL
31 exports.getAllMovies = async (req, res) => {
32   try {
33     const pool = getPool();
34     const [rows] = await pool.query('SELECT * FROM movies ORDER BY id DESC');
35     res.json(rows);
36   } catch (err) {
37     res.status(500).json({ error: err.message });
38   }
39 };
40
41 // READ by ID
42 exports.getMovieById = async (req, res) => {
43   try {
44     const pool = getPool();
45     const [rows] = await pool.query('SELECT * FROM movies WHERE id = ?', [req.params.id]);
46
47     if (rows.length === 0) return res.status(404).json({ error: 'Movie tidak ditemukan' });
48
49     res.json(rows[0]);
50   } catch (err) {
51     res.status(500).json({ error: err.message });
52   }
53 };
```

Edit
src/controllers/
movieController.
js

```
LatExpressRawMySQLPromiseAsync > src > controllers > JS movieController.js > ...
54
55 // UPDATE
56 exports.updateMovie = async (req, res) => {
57   try {
58     const pool = getPool();
59     const { title, year } = req.body;
60
61     if (!title || !year) {
62       return res.status(400).json({ error: 'Title dan Year wajib diisi' });
63     }
64
65     const [result] = await pool.query(
66       'UPDATE movies SET title = ?, year = ?, updated_at = NOW() WHERE id = ?',
67       [title, year, req.params.id]
68     );
69
70     if (result.affectedRows === 0) return res.status(404).json({ error: 'Movie tidak ditemukan' });
71
72     const [rows] = await pool.query('SELECT * FROM movies WHERE id = ?', [req.params.id]);
73     res.json(rows[0]);
74   } catch (err) {
75     res.status(500).json({ error: err.message });
76   }
77 };
```

Edit
src/controllers/
movieController.
js

LatExpressRawMySQLPromiseAsync > src > controllers > JS movieController.js > ...

```
78
79 // DELETE
80 exports.deleteMovie = async (req, res) => {
81   try {
82     const pool = getPool();
83     const [result] = await pool.query('DELETE FROM movies WHERE id = ?', [req.params.id]);
84
85     if (result.affectedRows === 0) return res.status(404).json({ error: 'Movie tidak ditemukan' });
86
87     res.json({ message: 'Movie berhasil dihapus' });
88   } catch (err) {
89     res.status(500).json({ error: err.message });
90   }
91 };
92
```

KESIMPULAN

- **mysql** → lama, hanya callback (seperti di kode pertama kita tadi).
- **mysql2** → lebih modern, kompatibel dengan mysql, mendukung callback + Promise.
- **mysql2/promise** → versi khusus untuk Promise/async-await, tanpa callback sama sekali.

Kesimpulan

- **Promise ada di balik method bawaan mysql2/promise.**
- Kita **tidak perlu bikin Promise manual** (`new Promise((resolve, reject) => {...})`) karena driver sudah otomatis mengembalikannya.
- `await` hanya cara lain menulis `.then().catch()` supaya lebih mudah dibaca.

TUGAS

Petunjuk Pengerjaan

CATATAN : SEKALI LAGI TIDAK PERLU BUAT REPOSITORY BARU (GUNAKAN REPISITORY TUGAS)

Ikutilah langkah-langkah pengerjaan di bawah ini:

1. Menambahkan Folder baru dan file baru

Gunakanlah repository tugas (jangan buat repository baru lagi). Lalu buatlah 1 folder baru dengan nama tambahkan folder baru dengan nama **"tugas-rest-api-mysql"**

2. Kerjakan Soal di bawah ini

kerjakan soal di bawah ini didalam folder **tugas-rest-api-mysql**

buatlah tabel nilai_mahasiswa dengan spesifikasi seperti dibawah ini

```
id INT PRIMARY KEY AUTO_INCREMENT
nama VARCHAR(192) NOT NULL
mata_kuliah VARCHAR(192) NOT NULL
nilai INT NOT NULL
indeks_nilai VARCHAR(4) NOT NULL
created_at DATETIME DEFAULT NOW()
updated_at DATETIME DEFAULT NOW()
```

lalu buatlah CRUD REST API dengan Raw MySQL, untuk POST nilai mahasiswa yang diperbolehkan di input dalam **body json** hanya **nama, mata_kuliah dan nilai**

lalu **nilai** hanya bisa diinput dari 0 s/d 100, jika diisi dibawah 0 atau diatas 100 maka akan ada pesan data nilai salah

untuk indeks_nilai didapatkan dari function yang ada dalam kode backend dengan ketentuan

```
nilai >= 80 indeksnya A,
nilai >= 70 dan nilai < 80 indeksnya B,
nilai >= 60 dan nilai < 70 indeksnya c,
nilai >= 50 dan nilai < 60 indeksnya D,
nilai < 50 indeksnya E
```

lampirkan juga file sql dari database yang digunakan pada tugas ini



KEMNAKER

Kesimpulan

- Koneksi database dengan MySQL
- CRUD REST API
 - Create
 - Read
 - Update
 - Delete

Terima Kasih

